

1. Logika cyfrowa

| Układ              | Formuła / sygnatura                                                 | Opis                                                                                                                  |
|--------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Sumatory           |                                                                     |                                                                                                                       |
| Półsumator         | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                             |
| Pełny sumator (FA) | $s = x \oplus y \oplus ci$<br>$co = x \& y \mid ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                                 |
| Sumator n-bitowy   | $n \times \text{FA}$                                                | Kaskada (ripple-carry): wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to <b>Carry Out</b> całości. |

|                     |                         |                                                                                                                          |
|---------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Układy kombinacyjne |                         |                                                                                                                          |
| Dekoder             | $n \rightarrow 2^n$     | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest <b>tylko</b> bit nr $k$ .                                           |
| Multiplekser        | $2^n + n \rightarrow 1$ | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjście przechodzi $S$ -ty bit danych.                               |
| ALU                 | $A, B, f \rightarrow C$ | Dekoder na bitach $f$ wybiera operację (np. $A + B$ , $A \mid B$ , $A \& B$ ); multipleksowanie wyników bramkami AND/OR. |

|                             |                                  |                                                                                                                                                    |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Układy sekwencyjne (pamięć) |                                  |                                                                                                                                                    |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Zbudowany z dwóch sprzężonych <b>NOR</b> -ów. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1       | Wejścia <b>S</b> / <b>R</b> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <b>CLK=1</b> .                                                |
| Sterowany zboczem           | zbocze 1 $\rightarrow$ 0         | Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w <b>momencie</b> zmiany zegara.           |

2. Typy danych

| Typ    | char | short | int | long | float | double | void* |
|--------|------|-------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4   | 8    | 4     | 8      | 8     |

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2).  
Stąd **B2U** = bity  $\rightarrow$  unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

| Stała | Bity          | Wartość                          |
|-------|---------------|----------------------------------|
| UMax  | <b>11...1</b> | $2^w - 1$                        |
| TMax  | <b>01...1</b> | $2^{w-1} - 1$ ( <b>INT_MAX</b> ) |
| TMin  | <b>10...0</b> | $-2^{w-1}$ ( <b>INT_MIN</b> )    |
| −1    | <b>11...1</b> | te same bity co UMax             |

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu ( $< > ==$ )  $\rightarrow$  **int** promowany do **unsigned** (bity bez zmian,  $-1 \rightarrow \text{UMax}$ ).

4. Rozszerzanie, obcinanie i przesunięcia

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| Rozszerz. 0 ( <b>zext</b> )    | <b>unsigned</b> : dopisuje 0 z góry           |
| Rozszerz. znak ( <b>sext</b> ) | <b>signed</b> : powiela bit znaku (MSB)       |
| $x \ll k$                      | $x \cdot 2^k$ (sgn./uns.)                     |
| $u \gg k$                      | $\lfloor u/2^k \rfloor$ — logiczne (zera)     |
| $x \gg k$                      | arytmetyczne (kopia MSB), zaokr. do $-\infty$ |
| $x/2^k$ do 0                   | $(x + (1 \ll k) - 1) \gg k$                   |

Strength red.: **x\*24**  $\rightarrow$  **(x<<5)-(x<<3)**.

5. Operacje, overflow i endianness

|                                 |                                                           |
|---------------------------------|-----------------------------------------------------------|
| UAdd/TAdd                       | $(u + v) \bmod 2^w$                                       |
| Nadmiar (+)                     | $u, v > 0$ , a wynik $< 0$ (wynik $-2^w$ )                |
| Nadmiar (-)                     | $u, v < 0$ , a wynik $\geq 0$ (wynik $+2^w$ )             |
| Negacja U2                      | <b>-x</b> = <b>~x+1</b> ; <b>-TMin=TMin</b> , <b>-0=0</b> |
| Wykr. nadmiaru ( <b>s=x+y</b> ) | <b>((s^x)&amp;(s^y))&gt;&gt;(w-1)</b>                     |
| Maska / abs                     | <b>m=x&gt;&gt;31; abs=(x^m)-m</b>                         |

| Schemat       | Najniższy adres | <b>0x0123</b>   |
|---------------|-----------------|-----------------|
| Big Endian    | MSB             | <b>[01][23]</b> |
| Little Endian | LSB             | <b>[23][01]</b> |

6. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \qquad \text{Bias} = 2^{k-1} - 1$$

| Format | bity | s | exp (k) | frac (n) |
|--------|------|---|---------|----------|
| float  | 32   | 1 | 8       | 23       |
| double | 64   | 1 | 11      | 52       |

Kategorie wartości

| Kategoria      | exp                                  | Własności                                                                                                                                                          |
|----------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $\neq 0 \dots 0$<br>$\neq 1 \dots 1$ | $E = \text{exp} - \text{Bias}$ . $M = 1.\text{frac}$ .<br>Najgęściej ułożone wokół 0.                                                                              |
| Zdenormaliz.   | $= 0 \dots 0$                        | $E = 1 - \text{Bias}$ . $M = 0.\text{frac}$ .<br>Reprezentują +0.0 i -0.0 (frac = 0)<br>oraz liczby bardzo bliskie zera.                                           |
| Specjalne      | $= 1 \dots 1$                        | Gdy $\text{frac} = 0$ , to $+\infty$ lub $-\infty$<br>(nadmiar/dzielenie przez 0).<br>Gdy $\text{frac} \neq 0$ , to NaN (np. $\sqrt{-1}$ ,<br>$\infty - \infty$ ). |

Zaokrąglenie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).



G - ostatni zachowany, R - pierwszy usunięty, S - OR reszty

| Warunek  | Ułamek | Akcja                                       |
|----------|--------|---------------------------------------------|
| R=0      | < 0.5  | W dół (odcięcie)                            |
| R=1, S=1 | > 0.5  | W górę (+1 do G)                            |
| R=1, S=0 | = 0.5  | Do parzystej: w górę gdy G=1, w dół gdy G=0 |

Własności matematyczne i rzutowanie

- **Brak łączności:**  $(a + b) + c \neq a + (b + c)$ , przez zaokrąglenia i utratę danych.
- **Brak rozdzielności:**  $a(b + c) \neq ab + ac$ .
- **Rzutowanie int → float:** Może stracić precyzję (zaokrągła zgodnie z trybem).
- **Rzutowanie int → double:** Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- **Rzutowanie float/double → int:** Obcina ułamek w stronę 0. Jeśli poza zakresem lub NaN → z reguły zwraca Tmin (0x80000000).

7. Rejestry x86\_64

|      |                      |      |                      |
|------|----------------------|------|----------------------|
| %rax | %eax<br>%ah %al      | %rbx | %ebx<br>%bh %bl      |
| %rcx | %ecx<br>%ch %cl      | %rdx | %edx<br>%dh %dl      |
| %rsi | %esi<br>%sil         | %rdi | %edi<br>%dil         |
| %rbp | %ebp<br>%bp1         | %rsp | %esp<br>%spl         |
| %r8  | %r8d<br>%r8w<br>%r8b | %r9  | %r9d<br>%r9w<br>%r9b |

Uwaga: Rejestry od %r10 do %r15 używają schematu:  
%r10, %r10d, %r10w, %r10b.

Podział ról rejestrów

|                                  |                                                                                |
|----------------------------------|--------------------------------------------------------------------------------|
| %rax                             | Akumulator. Główny rejestr arytmetyczny. Przechowuje <b>wartość zwracaną</b> . |
| %rdi, %rsi, %rdx, %rcx, %r8, %r9 | Kolejno: <b>od 1. do 6. argumentu funkcji</b> . Caller-saved.                  |
| %rsp                             | Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.                               |
| %rbp                             | Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.                        |
| %rbx, %r12-%r15                  | Ogólnego przeznaczenia. Wszystkie <b>callee-saved</b> .                        |
| %rip                             | Wskaźnik instrukcji (Program Counter).                                         |

Rejestry wektorowe (zmiennoprzecinkowe)

|                 |                                                                                                         |
|-----------------|---------------------------------------------------------------------------------------------------------|
| %xmm0 do %xmm15 | 128-bitowe rejestry. %xmm0 to wartość zwracana (float / double). Pierwsze 8 to argumenty. Caller-saved. |
| %ymm0 do %ymm15 | 256-bitowe rozszerzenie rejestrów XMM. Dzielą z nimi najmłodsze 128 bitów.                              |

8. Tryby adresowania (operands)

| Tryb                   | Format         | Obliczanie adresu |
|------------------------|----------------|-------------------|
| Natychniastowy (Imm)   | \$Imm          | Imm               |
| Rejestrowy (Reg)       | %Ra            | %Ra               |
| Bezpośredni (Mem)      | Imm            | MI[Imm]           |
| Pośredni (Mem)         | (%Rb)          | MI[%Rb]           |
| Z przesunięciem        | D(%Rb)         | MI[%Rb + D]       |
| Skalowany (Ind/scaled) | D(%Rb, %Ri, S) | MI[%Rb+%Ri*S+D]   |
| Skalowany (baseless)   | (, %Ri, S)     | MI[%Ri * S]       |

Legenda: Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko: 1, 2, 4 lub 8).

## 9. Flagi stanu i sterowanie

|    |                                                               |
|----|---------------------------------------------------------------|
| ZF | Zero. Wynik to 0 (np. argumenty są równe).                    |
| SF | Sign. Wynik jest ujemny (MSB = 1).                            |
| CF | Carry. Przepełnienie dla liczb <b>bez znaku</b> (unsigned).   |
| OF | Overflow. Przepełnienie dla liczb <b>ze znakiem</b> (signed). |

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

| Sufiks               | Znaczenie                               |
|----------------------|-----------------------------------------|
| <code>e / z</code>   | Equal / Zero (równe / wynik to 0)       |
| <code>ne / nz</code> | Not Equal / Not Zero (nierówne)         |
| <code>s</code>       | Sign (wynik ujemny, <code>SF=1</code> ) |

### Liczby ze znakiem (signed)

|                     |                            |
|---------------------|----------------------------|
| <code>g / ge</code> | Greater / Greater or Equal |
| <code>l / le</code> | Less / Less or Equal       |

### Liczby bez znaku (unsigned)

|                     |                                   |
|---------------------|-----------------------------------|
| <code>a / ae</code> | Above / Above or Equal (No Carry) |
| <code>b / be</code> | Below / Below or Equal (Carry)    |

Translacja struktur kontrolnych (C → ASM)

- if-else:** `jX` (skok) lub `cmovX` (liczy obie ścieżki, unika kar za predykcję).  
**Uwaga:** `cmovX` psuje się przy drogich operacjach, wyłuskaniu wskaźników (`*p`) i efektach ubocznych (`x++`).
- switch:** Tablice skoków → czas  $O(1)$ . Skok pośredni: `jmp *.L4(,%rdi,8)`. Brak `break` ⇒ **fall-through**.
- Pętla for:** Zawsze redukowana do pętli **while**:  
`init; while(cond) { body; update; }`

Wzorce asemblerowe dla pętli:

| do-while                                      | while (Jump-to-mid)                                    | while (Guarded)                                               |
|-----------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------|
| <code>loop:</code><br>Body<br>if(T) goto loop | goto test<br>loop:<br>Body<br>test:<br>if(T) goto loop | if(!T) goto done<br>loop:<br>Body<br>if(T) goto loop<br>done: |

## 10. Procedury i stos

Stos w x86-64 rośnie **w dół** (w stronę niższych adresów). Rejestr `%rsp` wskazuje na **wierzchołek** stosu (najniższy zajęty adres). Wartości odkłada się używając `push` (zmniejsza `%rsp` o 8), a zdejmując `pop` (zwiększa `%rsp` o 8).

Przekazywanie sterowania

- `call dest`: Odkłada adres powrotu (kolejnej instrukcji) na stos i robi skok do `dest`.
- `ret`: Zdejmuje adres powrotu ze stosu i robi pod niego skok.

## 11. Konwencja Wywoływań (System V ABI)



Argumenty 7+: Na stosie od końca. Wynik w `%rax`.

Rejestry caller-saved vs callee-saved

| Caller-saved                                                                                                                                | Callee-saved                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| (Wołający musi zapisać)<br>Mogą zostać nadpisane w funkcji. By je zachować, caller kładzie je na stos przed <code>call</code> .             | (Wołany musi przywrócić)<br>Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed <code>ret</code> . |
| <code>%rax</code> , <code>%rdi</code> , <code>%rsi</code> ,<br><code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> - <code>%r11</code> | <code>%rbx</code> , <code>%rbp</code> , <code>%r12</code> - <code>%r15</code>                                        |

Ramka stosu

Każde wywołanie funkcji otrzymuje własną ramkę na stosie (dzięki temu **rekurencja** działa naturalnie). **Alokacja ramki jest potrzebna, gdy:** brakuje rejestrów na zmienne (spilling), użyto lokalnej tablicy/struktury, lub pobrano adres zmiennej lokalnej (operator `&`).

| Prolog                                                                                                                                                              | Epilog                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pushq %rbp</code> ;<br>zapisz stary base ptr<br><code>movq %rsp, %rbp</code> ;<br>nowy base ptr = rsp<br><code>subq \$32, %rsp</code> ;<br>alokacja 32 bajtów | <code>addq \$32, %rsp</code> ;<br>zwolnij alokację<br><code>popq %rbp</code> ;<br>przywróć base ptr<br><code>ret</code> ;<br>skok powrotny |

**Sprzątanie przez `leave`:** Zastępuje `movq %rbp, %rsp` i `popq %rbp` w jednej instrukcji.

## 12. Architektura CPU

### Fazy przetwarzania instrukcji

Fetch → Decode → Execute → Memory → Write

### Pipelining i hazardy

Nakładanie faz na siebie znacznie zwiększa throughput. Prowadzi to jednak do konfliktów (hazardów):

|            |                                                                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Danych     | Odczyt po zapisie. Wynik instrukcji nie jest w rejestrze, a kolejna już go potrzebuje.<br><b>Rozwiązanie:</b> Forwarding/bypassing (przekazanie z ALU prosto na wejście następnej operacji) lub wstrzymanie (stall/bubble). |
| Sterowania | Instrukcja skoku wymusza odgadnięcie następnego adresu.<br><b>Rozwiązanie:</b> Branch prediction. Błąd kosztuje wyczyszczenie potoku (branch misprediction penalty).                                                        |

### Out-of-Order

- Nowoczesne procesory nie wykonują instrukcji sekwencyjnie.
- Superskalarność:** Zdolność do wykonania wielu instrukcji w jednym cyklu zegara (posiadanie wielu jednostek wykonawczych).
  - Register renaming:** Dynamiczne mapowanie rejestrów logicznych (`%rax`) na wiele rejestrów fizycznych. Eliminuje fałszywe zależności (nadpisywanie tego samego rejestru).
  - Reorder buffer:** Gwarantuje, że instrukcje, choć liczone asynchronicznie, są ostatecznie zatwierdzane w oryginalnej kolejności programu, by zachować spójność.

## 13. Tablice i struktury

### Reprezentacja tablic w pamięci

| Typ            | Deklaracja              | Adres                                                                                                  |
|----------------|-------------------------|--------------------------------------------------------------------------------------------------------|
| Jednowymiarowa | <code>T A[IL]</code>    | $A[i] = x_A + i \times \text{sizeof}(T)$                                                               |
| Wielowymiarowa | <code>T A[R1][C]</code> | $A[i][j] = x_A + (i \times C + j) \times \text{sizeof}(T)$                                             |
| Wielopoziomowa | <code>T *A[IL]</code>   | Adres elementu: $M[x_A + i \times 8] + j \times \text{sizeof}(T)$<br>(Wymaga dwóch odczytów z pamięci) |

### Wyrównanie danych i padding

- Zasada ogólna:** Obiekt o rozmiarze  $K$  bajtów musi znajdować się pod adresem podzielnym przez  $K$ , czyli  $p \bmod K = 0$ .
- Wyrównanie struktur:** Każde pole struktury jest wyrównywane do własnego rozmiaru  $K$ . Łączny rozmiar całej struktury musi być podzielny przez największe wewnętrzne wyrównanie. W razie potrzeby kompilator dodaje padding na końcu.
- Optymalizacja:** Układanie pól w strukturze od największego do najmniejszego minimalizuje marnowanie pamięci na padding.

## 14. Układ pamięci

### Mapa pamięci procesu

|             |                                                                                |
|-------------|--------------------------------------------------------------------------------|
| Stack       | Rośnie w dół. Zmienne lokalne, adresy powrotu (limit 8MB).                     |
| Shared libs | Współdzielone biblioteki (np. <code>libc.so</code> ).                          |
| Szterta     | Rośnie w górę. Dynamiczna alokacja ( <code>malloc</code> , <code>new</code> ). |
| Data        | Zmienne globalne i statyczne (zainicjowane i niezainicjowane).                 |

|      |                                              |
|------|----------------------------------------------|
| Text | Read-only. Binarny kod wykonywalny programu. |
|------|----------------------------------------------|

### Zagrożenia i mechanizmy obronne

|                 |                                                                                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Buffer overflow | Brak sprawdzania granic tablic. Nadpisuje zawartość stosu (np. stary <code>%rbp</code> i adres powrotu), pozwalając na przejęcie sterowania.                                        |
| ROP (Gadżety)   | Return-Oriented Programming. Wykorzystanie legalnych, krótkich skrawków kodu zakończonych <code>ret</code> do ominięcia blokady wykonywania.                                        |
| Stack canaries  | Losowa wartość wstawiana tuż przed adresem powrotu. Przed <code>ret</code> weryfikowana instrukcjami <code>xor</code> i <code>je</code> . Jeśli uszkodzona → <code>abort()</code> . |
| NX / ASLR       | <b>NX:</b> Stos i szterta dostają zakaz wykonywania kodu.<br><b>ASLR:</b> Losowanie bazowych adresów obszarów pamięci przy każdym starcie.                                          |

### Unie

Wszystkie pola unii współdzielą **ten sam adres początkowy** (offset 0). Alokacja równa jest rozmiarowi **największego elementu**. Używane głównie do manipulacji na poziomie bitów z ominięciem systemu typów (np. odczyt bitów typu `float` jako `int`).

## 15. Optymalizacje i ich ograniczenia

Kompilator nie zoptymalizuje kodu, jeśli istnieje szansa, że zmieni to zachowanie programu (np. dla specyficznych argumentów).

### Optymalizacyjne blokady

|                      |                                                                                                                                                                                                                       |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aliasing pamięci     | Dwa wskaźniki (np. <code>*p</code> , <code>*q</code> ) mogą wskazywać na to samo. Kompilator nie schowa sumy w rejestrze, tylko wymusza odczyt/zapis do RAM. Można wykorzystać słowo kluczowe <code>restrict</code> . |
| Wywołania funkcji    | Funkcja w pętli może mieć ukryte efekty uboczne (np. licznik globalny). Kompilator jej nie wyrzuci poza pętlę. <b>Rozwiązanie:</b> Inlining (wklejenie kodu) lub makra.                                               |
| Arytmetyka float -ow | Brak łączności: $(a + b) + c \neq a + (b + c)$ . Kompilator <b>nigdy</b> nie zmieni kolejności działań na <code>float</code> -ach w trosce o utratę precyzji/zaokrąglenia.                                            |

### Podstawowe techniki optymalizacji

|                    |                                                                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code motion        | Wyrzucenie obliczeń niezmienników (np. <code>x * y</code> ) całkowicie poza pętlę.                                                                                           |
| Strength reduction | Zamiana kosztownych instrukcji na tańsze (np. <code>x * 64</code> → <code>x &lt;&lt; 6</code> ), lub iterowanie za pomocą wskaźnika zamiast mnożenia indeksu przez rozmiar). |
| Share subexpr.     | Wykrywanie i jednokrotne obliczanie powtarzających się fragmentów równań.                                                                                                    |
| Loop unrolling     | Rozwinięcie pętli (np. przeskoczenie co 2 iteracje). Zmniejsza narzut związany z instrukcjami pętli i pozwala procesorowi na równoległość.                                   |

## 16. Linkowanie i konsolidacja

### Fazy budowania programu

```
cpp (Preprocessor) → cc1 (Kompilator) → as (Assembler) → ld (Linker)
```

### Formaty plików obiektowych (ELF)

- **Relokowalne ( .o )**: Kod i dane gotowe do połączenia z innymi plikami .o .
- **Wykonywalne ( a.out )**: Sklejone, gotowe do załadowania do pamięci i uruchomienia.
- **Współdzielone ( .so )**: Linkowane dynamicznie podczas ładowania lub działania.

|                   |                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------|
| .text             | Skompilowany kod maszynowy.                                                                      |
| .rodata           | Dane tylko do odczytu (np. "Witaj" w printf("Witaj"), tablice switch).                           |
| .data             | Zainicjowane zmienne globalne i statyczne (np. int x = 5;).                                      |
| .bss              | Niezainicjowane zmienne globalne/statyczne, nie zajmują miejsca w pliku.                         |
| .symtab           | Tablica symboli (informacje o funkcjach i zmiennych globalnych).                                 |
| .rel.text / .data | Informacje dla linkera, gdzie i jak wstawić ostateczne adresy w kodzie/danych podczas relokacji. |

### Symbol resolution

Linker rozróżnia 3 typy symboli:

- **Globalne**: zdefiniowane tu, używane tam,
- **Zewnętrzne**: extern, używane tu, zdefiniowane gdzie indziej, oraz
- **Lokalne**: z C-owym słowem static.

**Uwaga:** Zmienne lokalne na stosie **NIE** są w ogóle widoczne dla linkera

**Silne (Strong):** Funkcje i zainicjowane zmienne globalne (np. int foo = 5; ).

**Słabe (Weak):** Niezainicjowane zmienne globalne (np. int foo; ).

- Reguła 1** Wiele silnych symboli → **błąd linkowania** (Multiple definition).
- Reguła 2** 1 silny + n słabych → linker wybiera **silny** (słabe podpinają się pod niego).
- Reguła 3** Wiele słabych → linker wybiera **dowolny** (gcc -fno-common zakazuje takich sytuacji).

### Relokacja

Po rozwiązaniu symboli linker łączy sekcje .text / .data z wielu plików .o w jeden wielki blok. Przydziela im finalne adresy (run-time), a następnie modyfikuje wszystkie odniesienia w kodzie maszynowym do zmiennych i funkcji na podstawie wpisów z .rel.

### Biblioteki

#### Statyczne ( .a )

Zbiór plików .o. Linker kopiuje do pliku wykonywalnego **tylko te moduły**, które są faktycznie używane. Wada jest to, że każda aplikacja ma własną kopię w RAM. Zmiana wymusza

#### Dynamiczne ( .so )

Kod ładowany do pamięci raz. Pliki wykonywalne otrzymują tylko adresy podczas działania (przez GOT). W porównaniu do .a znacznie mniejsze pliki binarne, łatwe aktualizacje.

rekompilację.

**Kolejność flag gcc :**

Biblioteki .a podaje się na samym końcu komendy.

17. Katalog instrukcji (AT&T)

| Opcode                      | Efekt                         | Opis                                                          |
|-----------------------------|-------------------------------|---------------------------------------------------------------|
| <code>mov S, D</code>       | $D \leftarrow S$              | Kopiuje wartość z S do D.                                     |
| <code>lea S, D</code>       | $D \leftarrow \text{addr}(S)$ | Oblicza adres S (bez czytania pamięci).                       |
| <code>add S, D</code>       | $D \leftarrow D + S$          | Dodawanie (ustawia flagi).                                    |
| <code>sub S, D</code>       | $D \leftarrow D - S$          | Odejmowanie (ustawia flagi).                                  |
| <code>imul S, D</code>      | $D \leftarrow D * S$          | Mnożenie liczb ze znakiem.                                    |
| <code>sal / shl k, D</code> | $D \ll= k$                    | Przesunięcie w lewo (mnożenie przez $2^k$ ).                  |
| <code>sar k, D</code>       | $D \gg= k$                    | Arytmetyczne w prawo (dzielenie). <b>Kopiuje znak.</b>        |
| <code>shr k, D</code>       | $D \gg= k$                    | Logiczne w prawo. Dopełnia zerami.                            |
| <code>and / or S, D</code>  | $D \leftarrow D \& /   S$     | Operacje bitowe (ustawiają flagi).                            |
| <code>xor S, D</code>       | $D \leftarrow D \wedge S$     | Często używane jako <code>xor %rax, %rax</code> do zerowania. |

Porównania i sterowanie

|                          |                                                    |                                                       |
|--------------------------|----------------------------------------------------|-------------------------------------------------------|
| <code>cmp S1, S2</code>  | $S2 - S1$                                          | Ustawia flagi jak odejmowanie, nie zapisuje wyniku.   |
| <code>test S1, S2</code> | $S2 \& S1$                                         | Ustawia flagi jak <b>AND</b> .                        |
| <code>jX dest</code>     | $\text{if}(X) \text{\%rip} \leftarrow \text{dest}$ | Skok warunkowy (X = warunek).                         |
| <code>setX D</code>      | $\text{if}(X) D \leftarrow 1$                      | Ustawia najmłodszy bajt na 0 lub 1 na podstawie flag. |
| <code>cmovX S, D</code>  | $\text{if}(X) D \leftarrow S$                      | Warunkowe kopiowanie (optymalizacja zamiast skoku).   |

Stos i zarządzanie procedurami

|                        |                                                                           |                                                       |
|------------------------|---------------------------------------------------------------------------|-------------------------------------------------------|
| <code>push S</code>    | $\text{\%rsp} - = 8$<br>$M[\text{\%rsp}] \leftarrow S$                    | Odkłada na stos (zmniejsza <code>%rsp</code> ).       |
| <code>pop D</code>     | $D \leftarrow M[\text{\%rsp}]$<br>$\text{\%rsp} + = 8$                    | Zdejmuje ze stosu do D (zwiększa <code>%rsp</code> ). |
| <code>call dest</code> | push <code>%rip</code><br><code>%rip</code> $\leftarrow$ dest             | Skok do funkcji (zapisuje adres powrotu na stosie).   |
| <code>ret</code>       | pop <code>%rip</code>                                                     | Zdejmuje adres powrotu ze stosu i skacze pod niego.   |
| <code>leave</code>     | <code>%rsp</code> $\leftarrow$ <code>%rbp</code><br>pop <code>%rbp</code> | Sprząta ramkę stosu (odwrotność prologu).             |

Operacje wektorowe

Rozszerzenie AVX. Przedrostek `v` (nie niszczy źródła).

|                                    |                            |                                                                                                         |
|------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------|
| <code>vmovaps / ups S, D</code>    | $D \leftarrow S$           | Kopiuje wektor. <b>a</b> (aligned - szybkie, pamięć dzieli się przez wyrównanie), <b>u</b> (unaligned). |
| <code>vaddps / pd S1, S2, D</code> | $D \leftarrow S2 + S1$     | Dodawanie wielokrotne (packed). <b>ps</b> (single-float), <b>pd</b> (double).                           |
| <code>vmulps / pd S1, S2, D</code> | $D \leftarrow S2 * S1$     | Mnożenie wektorowe.                                                                                     |
| <code>vfmadd231ps S1, S2, D</code> | $D \leftarrow S2 * S1 + D$ | <b>Fused Multiply-Add</b> . Mnoży i dodaje do <b>D</b> w jednym kroku.                                  |

Operacje zmiennoprzecinkowe

|                                   |                                |                                                                                                                                  |
|-----------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>movsd / movss S, D</code>   | $D \leftarrow S$               | Kopiuje wartość skalarną ( <b>double</b> / <b>float</b> ) między rejestrami XMM lub pamięcią.                                    |
| <code>movapd / movaps S, D</code> | $D \leftarrow S$               | Kopiuje wyrównany wektor zmiennoprzecinkowy.                                                                                     |
| <code>addsd / subsd S, D</code>   | $D \leftarrow D \text{ op } S$ | Skalarne dodawanie / odejmowanie podwójnej precyzji.                                                                             |
| <code>xorpd / xorps S, D</code>   | $D \leftarrow D \wedge S$      | Bitowy XOR na rejestrach XMM. Używany jako <code>xorpd %xmm0, %xmm0</code> do szybkiego zerowania rejestru.                      |
| <code>ucomisd S1, S2</code>       | $S2 - S1$                      | Porównuje wartości typu <b>double</b> i ustawia odpowiednio flagi <b>ZF</b> , <b>PF</b> , <b>CF</b> w rejestrze <b>%rflags</b> . |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: **b** (8-bit), **w** (16-bit), **d** (32-bit), **q** (64-bit).  
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).